

PSI-SAPU Project Documentation

Generated for the current workspace state.

This document explains how the project is organized, what each file does, how the encoder -> TPSAPU backbone -> decoder pipeline works, and what the main numeric defaults mean.

High-Level Architecture

The project trains MNIST classifiers using a modular pipeline:

1. Encoder: converts an MNIST image shaped '(batch, 1, 28, 28)' into token features shaped '(batch, steps, embed_dim)'.
2. TPSAPU backbone: processes the token sequence through parallel LIF reservoirs with shared topology and different membrane time constants.
3. Decoder: reads membrane features, spike features, both, or full dynamics and produces 10 class logits.

The backbone is intentionally import-safe. Importing 'tpsapu.py' defines model classes only; it does not download data or start training.

Repository Files

'tpsapu.py'

Defines the reusable TPSAPU backbone.

Important classes and functions:

- * 'TPSAPUBackboneConfig': frozen dataclass with backbone defaults.
- * 'SharedReservoir': one LIF reservoir that owns neuron state but receives the shared input and recurrent weights from 'TPSAPUBackbone'.
- * 'TPSAPUBackbone': the reusable backbone module.
- * 'build_tpsapu_backbone': convenience factory.
- * 'TPSAPU': alias for 'TPSAPUBackbone'.

How it works:

- * Input tensor is normalized to '(batch, steps, input_dim)'.
- * 'nl_proj' maps encoder tokens into reservoir space.
- * Each timestep is sent to all reservoirs in parallel.
- * All reservoirs share:
 - 'shared_input_proj'
 - 'shared_recurrent'
- * Each reservoir has its own LIF node and tau.
- * The backbone records:
 - 'membrane': normalized membrane readout across all reservoirs.
 - 'spike': spike output across all reservoirs.
 - 'dynamics': first membrane value then membrane deltas.
 - 'spike_history': cumulative spike count per feature over time.

Magic numbers and defaults:

- * 'reservoir_dim=64': default neurons per reservoir.
- * 'taus=(1.1, 8.0, 64.0)': fast, medium, and slow LIF memory timescales.
- * 'recurrent_drop_p=0.0': no recurrent dropout unless requested.
- * 'input_hidden_dim=None': defaults to 'reservoir_dim * 2'.
- * 'v_threshold=1.0': LIF fires when membrane voltage reaches 1.
- * 'v_reset=0.0': LIF membrane resets to zero after spike.
- * 'detach_reset=True': reset path is detached for more stable surrogate gradient training.
- * 'surrogate.ATan()': differentiable surrogate for spike gradients.
- * recurrent weight init 'uniform(-0.001, 0.001)': small initial recurrent influence so the model starts stable.
- * 'output_norm=True': applies 'LayerNorm' to membrane concatenation.

Output states:

- * 'state="membrane"' returns membrane features.
- * 'state="spike"' returns spike features.
- * 'state="both"' returns membrane and spike concatenation.
- * 'state="all"' returns membrane, spike, dynamics, and spike history.

Pooling:

- * 'last': use the final timestep.
- * 'mean': average all timesteps.
- * 'none' or 'sequence': return all timesteps.

'train_pipeline.py'

Main training entry point.

Primary responsibilities:

- * Parses CLI configuration.
- * Builds MNIST dataloaders.
- * Builds encoder, TPSAPU backbone, and decoder.
- * Prints trainable, frozen, and total parameter counts before any training or resumed training starts.
- * Trains with warmup, cosine schedule, and L2 pruning.
- * Saves full checkpoints that can resume training.

Pipeline wrapper:

- * 'EncoderBackboneDecoder' calls:
 - 'encoder(images)'
 - 'backbone(tokens, state=decoder.input_state, pooling=...)'
 - 'decoder(features)'
- * If the decoder needs sequence input, pooling is forced to 'none'.

Training schedule defaults:

- * 'warmup_epochs=5': linear LR warmup.
- * 'cosine_epochs=35': main cosine annealing stage.

- * 'cosine_cycles=1.0': one cosine cycle by default.
- * 'prune_epochs=20': L2 pruning stage.
- * 'prune_cycles=1.0': one cosine cycle during pruning.
- * 'min_lr_ratio=0.05': cosine LR bottom is 5 percent of base LR.

Pruning defaults:

- * 'l2_prune_start_lambda=1e-7': first pruning L2 weight.
- * 'l2_prune_growth_epochs=4': multiply lambda by 10 every 4 pruning epochs.
- * 'prune_threshold=0.0': no fixed threshold by default; target sparsity drives the mask.
- * 'target_sparsity=0.95': keep only the largest-magnitude 5 percent of shared recurrent weights by the end of pruning.
- * 'prune_start_sparsity=0.0': ramp starts from dense recurrent topology.
- * 'prune_epochs=30': longer pruning phase for high sparsity.
- * 'prune_ramp_epochs=0': infer ramp length from prune epochs minus stabilization epochs.
- * 'prune_stabilize_epochs=6': final epochs train with the target mask fixed at 95 percent sparsity.
- * 'prune_lr_scale=0.2': pruning LR starts at 20 percent of base LR.

For the default 30 pruning epochs, L2 lambda is:

- * epochs 1-4: '1e-7'
- * epochs 5-8: '1e-6'
- * epochs 9-12: '1e-5'
- * epochs 13-16: '1e-4'
- * epochs 17-20: '1e-3'
- * epochs 21-24: '1e-2'
- * epochs 25-28: '1e-1'
- * epochs 29-30: '1e0'

The pruning mask uses a smooth target-sparsity ramp. With defaults, the first 24 pruning epochs gradually increase target sparsity toward 95 percent, then the final 6 epochs stabilize at 95 percent.

Optimization defaults:

- * 'batch_size=128': default MNIST batch size.
- * 'lr=1e-3': AdamW base learning rate.
- * 'weight_decay=1e-4': AdamW weight decay.
- * 'grad_clip=1.0': maximum global gradient norm.
- * 'seed=42': reproducibility seed.
- * 'num_workers=2': dataloader workers.

Dataset constants:

- * MNIST images are normalized with mean '0.1307' and std '0.3081'.
- * 'num_classes=10' is hard-coded in decoder construction because MNIST has 10 digits.

Checkpoint behavior:

- * '--checkpoint-out' writes a full latest checkpoint.
- * A sibling 'best.pt' is saved whenever validation accuracy improves.
- * '--resume' loads a latest checkpoint and continues from stored progress.
- * Checkpoints include:

- model state
- optimizer state
- args
- completed train/prune epoch counters
- validation metrics
- best metric and best epoch
- pruning mask
- Python, NumPy, Torch, and CUDA RNG states

Log behavior:

- * Logs default to the directory containing '--checkpoint-out'.
- * Override with '--log-dir'.
- * 'train.log' mirrors console output.
- * 'metrics.jsonl' stores one JSON object per epoch.
- * 'metrics.csv' stores the same metrics in spreadsheet-friendly form.
- * 'args.json' stores the run configuration used for the current run.
- * On resume, logs append instead of replacing earlier history.

Startup printout:

- * The script prints a parameter summary before the first training epoch.
- * The summary includes total trainable/frozen parameters and per-module counts for encoder, backbone, and decoder.

'inference.py'

Inference and visualization script.

Primary responsibilities:

- * Loads checkpoint architecture arguments.
- * Allows optional architecture overrides from CLI.
- * Builds the model and loads 'model_state'.
- * Runs a single MNIST image through the model.
- * Produces temporal visualizations.

Outputs:

- * *_timeline.png': image regions revealed over encoder timesteps with prediction and confidence per step.
- * *_confidence.png': top-class confidence over time.
- * *_activity.png': membrane and spike activity per tau over time.
- * *_timeline.gif': optional animation with '--gif'.

Magic numbers:

- * '28': MNIST image size.
- * '14 x 14': token grid for 2x2 LIF encoder.
- * '7 x 7': token grid for CNN encoders and default 7x7 patch encoder.
- * 'cell_size=2': reveal mask for 2x2 LIF encoder.
- * 'cell_size=4': reveal mask approximation for CNN token grids.
- * 'fps=2', 'interval=450': GIF playback speed and frame interval.

Temporal inference logic:

- * The script calls 'backbone.forward_states(tokens)' once.
- * It constructs the feature sequence required by the decoder:
 - membrane
 - spike
 - membrane+spike
 - all-state
- * For sequence decoders such as 'lif_count', it reruns the decoder on prefixes: first token, first two tokens, and so on.
- * For mean pooling, it uses cumulative means so each plotted step represents the information available up to that step.

'run_all_combinations.py'

Sweep runner for encoder/decoder combinations.

It runs every configured encoder with every configured decoder.

Default encoders:

- * 'linear_patch'
- * 'mlp_patch'
- * 'lif_2x2'
- * 'cnn2'
- * 'cnn3'
- * 'res_cnn'
- * 'rows'

Default decoders:

- * 'linear'
- * 'membrane_mlp'
- * 'spike_mlp'
- * 'both_mlp'
- * 'all_state_mlp'
- * 'lif_count'

Default cycles:

- * 'cycles=3.0'
- * Passed to both '--cosine-cycles' and '--prune-cycles'.

Output layout:

- * 'sweep_runs/<encoder>__<decoder>/latest.pt'
- * 'sweep_runs/<encoder>__<decoder>/best.pt'
- * 'sweep_runs/<encoder>__<decoder>/command.txt'
- * 'sweep_runs/<encoder>__<decoder>/train.log'
- * 'sweep_runs/<encoder>__<decoder>/metrics.jsonl'
- * 'sweep_runs/<encoder>__<decoder>/metrics.csv'
- * 'sweep_runs/<encoder>__<decoder>/args.json'

Resume behavior:

- * If 'latest.pt' exists, the script resumes it by default.

- * Use '--no-resume-existing' to force a fresh command.
- * Use '--skip-existing' to skip folders that already have 'latest.pt'.

'requirements.txt'

Lists runtime dependencies:

- * 'torch': PyTorch model and training.
- * 'torchvision': MNIST dataset and transforms.
- * 'spikingjelly': LIF neurons and surrogate gradients.
- * 'numpy': seeding and plotting arrays.
- * 'matplotlib': inference plots and GIF generation.
- * 'pillow': GIF writer backend.

'gitignore'

Ignores generated or local-only artifacts:

- * Python caches.
- * Pytest cache.
- * MNIST data.
- * checkpoints.
- * inference outputs.
- * sweep outputs.

'sapu-psi.zip'

Archive artifact present in the workspace. It is not used by the current Python pipeline and is not imported by any source file.

Encoder Files

'encoders/__init__.py'

Exports all encoder classes for convenient imports. It also keeps 'MNISTPatchEncoder' as an alias for 'LinearPatchEncoder'.

'encoders/common.py'

Shared encoder helpers:

- * 'validate_mnist_images': validates '(batch, channels, height, width)'.
- * 'init_pos_embed': trainable positional embedding initialized with truncated normal std '0.02'.
- * 'flatten_feature_map': converts CNN feature maps from '(B, C, H, W)' to '(B, H*W, C)'.

Magic numbers:

* std '0.02': common small positional embedding initialization.

'encoders/spiking.py'

Lazy SpikingJelly import helper for encoder modules. It lets files import cleanly, then raises a clear error only when a spiking encoder is constructed without SpikingJelly installed.

'encoders/linear_patch.py'

Linear patch projection encoder.

How it works:

- * Splits image into non-overlapping patches with 'F.unfold'.
- * Applies 'LayerNorm', one 'Linear', and dropout.
- * Adds trainable positional embedding.

Defaults and magic numbers:

- * 'image_size=28': MNIST.
- * 'patch_size=7': produces '4 x 4 = 16' tokens.
- * 'in_channels=1': grayscale.
- * 'embed_dim=128': token size for TPSAPU.
- * 'dropout=0.0': no encoder dropout unless requested.

'encoders/mlp_patch.py'

MLP patch projection encoder.

How it works:

- * Same patch extraction as 'linear_patch'.
- * Uses 'LayerNorm -> Linear -> GELU -> Dropout -> Linear -> Dropout'.
- * Adds positional embedding.

Defaults and magic numbers:

- * 'patch_size=7': 16 tokens.
- * 'hidden_dim=None': becomes 'max(embed_dim, patch_dim * 2)'.
- * For default 7x7 grayscale patches, 'patch_dim=49', so hidden default is 'max(128, 98) = 128'.

'encoders/row.py'

Row encoder.

How it works:

- * Squeezes channel dimension from '(B, 1, 28, 28)' to '(B, 28, 28)'.
- * Treats each image row as one token.
- * Applies 'LayerNorm(28) -> Linear(28, embed_dim) -> GELU -> Dropout'.

- * Adds one positional embedding per row.

Defaults and magic numbers:

- * 'image_size=28': 28 sequence steps.
- * 'in_channels=1': grayscale MNIST.

'encoders/lif_2x2.py'

Intensity-driven 2x2 LIF encoder.

How it works:

- * Denormalizes MNIST pixels back to '[0, 1]'.
- * Splits image into '2 x 2' patches, producing '14 x 14 = 196' tokens.
- * Computes mean patch intensity.
- * Gives bright patches an immediate current boost.
- * Runs one LIF step per patch token.
- * Adds positional embedding.

Defaults and magic numbers:

- * 'patch_size=2': local 2x2 pixel area.
- * 'white_threshold=0.6': patch is considered mostly white above this mean.
- * 'intensity_gain=1.2': scales continuous intensity drive.
- * 'immediate_boost=1.0': pushes white patches over the LIF threshold.
- * 'tau=2.0': encoder LIF membrane time constant.
- * 'v_threshold=1.0': spike threshold.
- * learned detail scale '0.1': keeps learned patch detail secondary to intensity drive.
- * MNIST denormalization uses mean '0.1307' and std '0.3081'.

'encoders/tiny_cnn2.py'

Two-layer tiny CNN encoder.

How it works:

- * Conv layer 1 downsamples from '28 x 28' to '14 x 14'.
- * Conv layer 2 downsamples from '14 x 14' to '7 x 7'.
- * Flattens the '7 x 7' feature map to 49 tokens.
- * Adds positional embedding.

Defaults and magic numbers:

- * 'kernel_size=3', 'stride=2', 'padding=1': standard downsampling conv.
- * 'hidden_channels=64': intermediate CNN width.
- * 'num_patches=49': 7x7 output grid.

'encoders/tiny_cnn3.py'

Three-layer tiny CNN encoder.

How it works:

- * First two convs downsample to '7 x 7'.
- * Third conv refines at '7 x 7' with stride 1.
- * Outputs 49 tokens.

Defaults and magic numbers:

- * Third conv uses 'stride=1' to preserve '7 x 7'.
- * 'num_patches=49'.

'encoders/residual_cnn.py'

Residual tiny CNN encoder.

How it works:

- * Conv1 and conv2 produce second-layer '7 x 7' features.
- * Conv3 refines those features.
- * Residual output is 'layer2 + conv3(layer2)'.
- * The sequence concatenates both second-layer tokens and residual third-layer tokens.

Defaults and magic numbers:

- * 'num_patches=98': two '7 x 7' token grids.
- * Residual addition preserves feature scale and lets decoder/backbone see both intermediate and refined representations.

Decoder Files

'decoders/__init__.py'

Exports all decoder classes and keeps 'MLPDecoder' as an alias for 'MembraneMLPDecoder'.

'decoders/common.py'

Shared helper:

- * 'mlp_layers': builds 'LayerNorm', repeated 'Linear -> GELU -> Dropout', and a final output 'Linear'.

Magic numbers:

- * minimum MLP 'depth=2': at least one hidden projection and one output projection.

'decoders/spiking.py'

Lazy SpikingJelly import helper for decoder modules. Used by 'lif_count.py'.

'decoders/linear.py'

Single linear classifier over pooled membrane features.

Metadata:

```
* 'input_state="membrane"'
* 'input_multiplier=1'
* 'needs_sequence=False'
```

Magic numbers:

```
* 'num_classes=10': MNIST digits.
```

'decoders/membrane_mlp.py'

Two-layer MLP over pooled membrane features.

Metadata:

```
* 'input_state="membrane"'
* 'input_multiplier=1'
* 'needs_sequence=False'
* 'MLPDecoder' aliases this class.
```

Defaults:

```
* 'hidden_dim=128'
* 'dropout=0.1'
* 'depth=2'
```

'decoders/spike_mlp.py'

Two-layer MLP over pooled spike features.

Metadata:

```
* 'input_state="spike"'
* 'input_multiplier=1'
* 'needs_sequence=False'
```

Defaults:

```
* 'hidden_dim=128'
* 'dropout=0.1'
* 'depth=2'
```

'decoders/membrane_spike_mlp.py'

Three-layer MLP over concatenated membrane and spike features.

Metadata:

```
* 'input_state="both"'
* 'input_multiplier=2'
* 'needs_sequence=False'
```

Defaults:

```
* input dimension is 'backbone.out_features * 2'.
* 'depth=3', because it combines two feature types.
```

'decoders/all_state_mlp.py'

Four-layer MLP over all available backbone state.

Input concatenates:

```
* membrane readout
* current spikes
* membrane dynamics
* cumulative spike history
```

Metadata:

```
* 'input_state="all"'
* 'input_multiplier=4'
* 'needs_sequence=False'
```

Defaults:

```
* input dimension is 'backbone.out_features * 4'.
* 'depth=4', because it combines four feature groups.
```

'decoders/lif_count.py'

Ten class LIF neurons with spike-count output.

How it works:

```
* Expects full sequence input.
* Projects each timestep to 10 class currents.
* Runs a class-level LIF node over time.
* Sums class spikes over the sequence.
* The spike counts are used as class logits.
```

Metadata:

```
* 'input_state="both"'
* 'input_multiplier=2'
* 'needs_sequence=True'
```

Magic numbers:

- * 'num_classes=10': one LIF neuron per MNIST digit.
- * 'tau=2.0': class LIF memory time constant.
- * 'v_threshold=1.0': spike threshold.
- * 'v_reset=0.0': reset value.

End-to-End Flow

Training command example:

```
python train_pipeline.py --encoder linear_patch --decoder membrane_mlp
```

Flow:

1. CLI args are parsed.
2. Seed is set.
3. MNIST loaders are built.
4. Encoder is selected from the encoder registry.
5. TPSAPU backbone is constructed.
6. Decoder is selected from the decoder registry.
7. The model trains through warmup and cosine phases.
8. Pruning phase applies L2 pressure and persistent magnitude masking.
9. 'latest.pt' is written after every epoch.
 10. 'best.pt' is written whenever validation accuracy improves.
 11. Logs and metrics are written beside the checkpoints.

Sweep command example:

```
python run_all_combinations.py
```

This runs 42 combinations: 7 encoders times 6 decoders.

Inference command example:

```
python inference.py --checkpoint sweep_runs/linear_patch__linear/latest.pt --index 0 --gif
```

Important Shape Conventions

- * Raw MNIST: '(batch, 1, 28, 28)'.
- * Encoder output: '(batch, steps, embed_dim)'.
- * Backbone membrane/spike output: '(batch, steps, reservoir_dim * num_taus)'.
- * 'both' decoder input: feature dimension doubled.
- * 'all_state_mlp' input: feature dimension quadrupled.
- * Classification logits: '(batch, 10)'.

Main CLI Defaults

Architecture:

- * '--encoder linear_patch'
- * '--decoder membrane_mlp'
- * '--pooling last'
- * '--embed-dim 128'
- * '--reservoir-dim 64'
- * '--taus 1.1,8.0,64.0'
- * '--patch-size 7'
- * '--cnn-channels 64'
- * '--lif-white-threshold 0.6'

Training:

- * '--warmup-epochs 5'
- * '--cosine-epochs 35'
- * '--cosine-cycles 1.0'
- * '--prune-epochs 30'
- * '--prune-cycles 1.0'
- * '--target-sparsity 0.95'
- * '--prune-stabilize-epochs 6'
- * '--batch-size 128'
- * '--lr 1e-3'
- * '--weight-decay 1e-4'
- * '--grad-clip 1.0'

Sweep:

- * '--cycles 3.0', applied to both cosine and pruning phases.
- * '--resume-existing' enabled by default.

Notes and Caveats

- * Spiking modules require SpikingJelly at construction time.
- * Importing modules without SpikingJelly is allowed where possible; errors are delayed until a spiking class is instantiated.
- * The inference reveal masks approximate CNN receptive fields as 7x7 cells.
- * 'sapu-psi.zip' is an archive artifact and not part of the active runtime.
- * 'data/', 'checkpoints/', 'sweep_runs/', and 'inference_outputs/' are ignored because they are generated artifacts.